

ECS401: Unit 9 Lab Progress Test

FEEDBACK: Answers and Explanations

Work hard and you will eventually be able to program. You should have dozens of small programs by now!

If you got any wrong you urgently need to do something to improve your understanding. There is only a little time left on the course.

1. Complete the dry run tables and give both the value ultimately returned and the text that is printed to the screen after a call $f(2)$; when f is defined as below.

```
public static int f(int k)    // L1
{
    if (k == 5)              // L2
    {
        System.out.print(k); // L3
        return 10;           // L4
    }
    else
    {
        System.out.print(k); // L5
        return (k + f(k+1)); // L6
    }
}
```

DRY RUN TABLES:

f(2)					
LINE	k	k == 5 ?	print	f(k+1)	return (k + f(k+1))
L1	2				
L2		2 == 5 ? false			
L5			2		
L6				f(3) =? 17	19

f(3)					
LINE	k	k == 5 ?	print	f(k+1)	return (k + f(k+1))
L1	3				
L2		3 == 5 ? false			
L5			3		
L6				f(4) =? 14	17

f(4)					
LINE	k	k == 5 ?	print	f(k+1)	return (k + f(k+1))
L1	4				
L2		4 == 5 ? false			
L5			4		
L6				f(5) =? 10	14

f(5)					
LINE	k	k == 5 ?	print	f(k+1)	return 10
L1	5				
L2		5 == 5 ? true			
L3			5		
L4					10

CORRECT ANSWER:

Printed: 2345

Returned: 19

EXPLANATION:

The call $f(2)$ sets k to 2. This leads to the else case being executed. 2 is printed and a call made to $f(k+1)$ is $f(3)$. Note this is moving towards the base case of 5. This call has to pause until it has a value for this recursive call - what is $f(3)$ equal to?

The call $f(3)$ is similar executing the else branch printing 3 and then doing a recursive call to $f(4)$. It pauses waiting for the answer...

The call $f(4)$ is similar again printing 5 and doing a recursive call to $f(5)$. It pauses waiting for the answer...

$f(5)$ is the base case so it prints 5 and then returns the answer 10...

We return to the call to $f(4)$. It adds 4 to the value returned from $f(5)$ i.e. it adds 4 to 10. This gives 14 as the value returned from $f(4)$.

We return to the call to $f(3)$. It adds 3 to the value returned from $f(4)$ i.e. it adds 3 to 14. This gives 17 as the value returned from $f(3)$.

We return to the original call to $f(2)$. It adds 2 to the value returned from $f(3)$ i.e. it adds 2 to 17. This gives 19 as the value returned from $f(2)$.

Note the printing is done before the calls so the values are printed on the way down through the recursion.

2. Complete the dry run table and give the text that is printed to the screen after a call. `printme(4)`; when `printme` is defined as below:

```
public static void printme(int n) // L1
{
    if (n == 0) // L2
    {
        System.out.print(1); // L3
        return; // L4
    }
    else
    {
        System.out.print(n+1); // L5
        return; // L6
    }
}
```

Fill out the dry run table below to work out the answer. HINT: this method is NOT recursive.

DRY RUN TABLE:

printme(4)				
LINE	n	n == 0 ?	print	return
L1	4			
L2		4 == 0 ? false		
L5			5	
				void

CORRECT ANSWER:

5

EXPLANATION:

This method does not include a recursive call. All it does is check if the value given is 0 (it isn't) and if not print 1 more than that number (stored in `n`). That means 5 is printed and then it returns.

3. Complete the dry run table and give the text that is printed to the screen after a call. `printme(4)`; when `printme` is defined as below:

```
public static void printme(int n) // L1
{
    if (n == 0) // L2
    {
        System.out.print(1); // L3
        return; // L4
    }
    else
    {
        System.out.print(n+1); // L5
        printme(n-2); // L6
        return; // L7
    }
}
```

DRY RUN TABLES:

printme(4)					
LINE	n	n == 0 ?	print	printme(n-2)	return
L1	4				
L2		4 == 0 ? false			
L5			5		
L6				printme(2)	
L7					void

printme(2)					
LINE	n	n == 0 ?	print	printme(n-2)	return
L1	2				
L2		2 == 0 ? false			
L5			3		
L6				printme(0)	
L7					void

printme(0)					
LINE	n	n == 0 ?	print	printme(n-2)	return
L1	0				
L2		0 == 0 ? true			
L3			1		
L4					void

CORRECT ANSWER:

531

EXPLANATION:

This method is recursive – it calls itself so acts like a loop, repeatedly calling itself and doing work until it hits the base case. The call is:

```
printme(4);
```

You pass it 4 and the 4 is stored in a new local variable n. That value is not 0 so it skips the base case and goes to the step (here the else case) case. The step case says you must first print 1 more than the value of n, so **5 is printed**:

```
System.out.print(n+1);
```

It then says call printme with the value of 2 less than n (i.e. 4-2 so 2) This means we can't move on until this has been done. So we pause what we are doing and do a new call to printme:

```
printme(2);
```

It creates a new variable n (that just happens to have the same name as the other variable but is a different variable holding a different value). This variable n has the value passed (2) in it. The method then checks if what's in n (2) is equal to 0. It isn't so it executes the step case:

```
System.out.print(n+1);
```

It involves printing the new copy of n that is local to this call. It holds value 2 so **3 is printed next**.

It then says call printme with the value of 2 less than n (i.e. 2-2 so 0). This means we can't move on until this has been done. So we pause what we are doing and do this new call to printme:

```
printme(0);
```

It creates a new running version of printme with a new local variable n that holds the 0. It tests whether this value is 0. It is and so it **prints 1** then returns:

```
return;
```

This returns control to the method that called it – another version of `printme` (the one with a `n` that has value 2). It wakes up and moves on from its call to `printme(0)` to its next instruction. The next instruction is `return`:

```
return;
```

so we return from this method to whoever did the call. It was another version of `printme` (the original call with `n` having value 4). It wakes up and moves on from its call to `printme(0)` to its next instruction. The next instruction is `return`:

```
return;
```

So we return from the method and are finished.

4. Complete the dry run table and give the text that is printed to the screen after a call. `printme(4)`; when `printme` is defined as below:

```
public static void printme(int n) // L1
{
    if (n == 0) // L2
    {
        System.out.print(1); // L3
        return; // L4
    }
    else
    {
        printme(n-2); // L5
        System.out.print(n+1); // L6
        return; // L7
    }
}
```

DRY RUN TABLES:

printme(4)					
LINE	n	n == 0 ?	printme(n-2)	print	return
L1	4				
L2		4 == 0 ? false			
L5			printme(2)		
L6				5	
7					void

printme(2)					
LINE	n	n == 0 ?	printme(n-2)	print	return
L1	2				
L2		2 == 0 ? false			
L5			printme(0)		
L6				3	
L7					void

printme(0)					
LINE	n	n == 0 ?	printme(n-2)	print	return
L1	0				
L2		0 == 0 ? true			
L3				1	
L4					void

CORRECT ANSWER:

135

EXPLANATION:

This method is recursive again so acts in a similar way to the previous method. However there is a difference. It does no printing BEFORE the recursive call. All the printing is done as it returns from the calls. The initial call is:

```
printme(4);
```

You pass it 4 and the `4` is stored in a new local variable `n`. That value is not 0 so it skips the base case and goes to the step case. The step case says you must first call `printme` with the value of 2 less than `n` (i.e. `4-2` so `2`):

```
printme(2);
```

Note we haven't printed anything yet and are now pausing the original call until we have finished this new call. A new version of `printme` is created with its new variable `n` that holds 2. It goes to the step case as `n` is not 0 and again the first thing is to pause while we call the new method:

```
printme(0);
```

This new call has a new variable `n` that holds 0. That means the base case is executed and **1 is printed** – the first thing that has been printed:

```
System.out.print(1);
```

The call to `printme(0)` returns: `return;` We are now back at the call to `printme(2)`. It moves on to its next instruction:
`System.out.print(n+1);`

It prints 1 more than the value in its `n`. 2 is in this variable `n`. That means **3 is printed**. Its next instruction says `return;`

We are now back at the call to `printme(4)` which had paused. It moves on to its next instruction:
`System.out.print(n+1);`

It prints 1 more than the value in its `n`. 4 is in this variable `n`. That means **5 is printed**. Its next instruction says `return;`

That returns to the original call so we are finished.

5. In a recursive function the idea is that on each step you leave a smaller, easier version of the problem in the sense that you move closer to the base case. In the method in question 4 above what is:

CORRECT ANSWER:

- a) The step case: the else case where `n` is not equal to 0.
- b) The base case: the case when `n` is equal to 0.